UNIVERSIDAD DE CALDAS

Módulo: Programación Web II

PARCIAL

Docente: Andrés Felipe Osorio Quintero

Unidad 2: Interactividad avanzada y almacenamiento local – Validación avanzada de formularios

Nombre del estudiante:	
Fecha:	

Instrucciones: lea con atención cada pregunta antes de responder.
Sección I: seleccione una sola opción por pregunta y márquela encerrando su letra en un círculo.
Secciones II y III: responda con sus propias palabras, de forma clara y argumentada, en las líneas dispuestas bajo cada pregunta.

SECCIÓN I. SELECCIÓN MÚLTIPLE

Preguntas 1 a 20 · Seleccione la única opción correcta (A, B, C o D).

1. Un campo `<input type="email" required>` contiene el valor `ana@` cuando la persona intenta enviar el formulario. ¿Qué describe correctamente el estado de su objeto `validity`?

   A. `valueMissing` es `true`, porque el texto incompleto se trata como si el campo estuviera vacío
   B. `typeMismatch` es `true`, `valueMissing` es `false` y, por tanto, `valid` también es `false`
   C. `patternMismatch` es `true`, porque el formato del correo se evalúa como una expresión regular
   D. `customError` es `true`, porque el navegador asigna por sí mismo un mensaje propio al campo

Editar con WPS Office

2. Un campo de contraseña declara `minlength="8"` y llega desde el HTML con el valor inicial `abc`, que la persona aún no ha modificado. Según el documento, ¿qué ocurre con la propiedad `tooShort`?

   A. Es `true` desde la carga, porque el valor ya incumple el mínimo declarado en el atributo
   B. Es `true`, pero el navegador oculta la burbuja hasta que se llama a `reportValidity()`
   C. Pasa a `true` en cuanto se dispara el evento `blur`, aunque el valor no haya cambiado
   D. Sigue en `false` mientras no haya edición de la persona, porque esa restricción se evalúa tras la edición

3. Un equipo desactiva la burbuja automática con `novalidate` en el formulario y, en un manejador propio, llama a `campo.checkValidity()` sobre un control `required` que está vacío. ¿Qué sucede?

   A. Devuelve `false` y dispara el evento `invalid` en ese control, sin mostrar la burbuja del navegador
   B. No ocurre nada, porque `novalidate` desactiva también la API de validación de restricciones
   C. Devuelve `false`, muestra la burbuja del navegador y entrega el foco al control inválido
   D. Devuelve `true`, porque con `novalidate` los controles dejan de participar en la validación

4. Un desarrollador registra `formulario.addEventListener('invalid', mostrarError)` para reaccionar ante cualquier control inválido, pero el manejador nunca se ejecuta. ¿Cuál es la causa y la corrección adecuada?

   A. El evento solo se dispara con `reportValidity()`; debe llamarse a ese método antes de cada envío
   B. El evento solo existe en controles `required`; debe sustituirse por el evento `input` del formulario
   C. El evento `invalid` no burbujea; debe registrarse en la fase de captura pasando `true` como tercer argumento
   D. El atributo `novalidate` impide que el evento se dispare; debe retirarse del formulario por completo

5. Un script asigna `campo.setCustomValidity('Usuario no disponible')`. La persona corrige el valor, pero el campo sigue marcándose como inválido y el formulario no se envía. ¿Cuál es la causa y la solución?

   A. `validity.valid` no se recalcula por sí sola; basta con llamar a `checkValidity()` después de cada corrección
   B. El estado personalizado no se borra solo; debe limpiarse con `setCustomValidity("")` antes de reevaluar las reglas
   C. `customError` se desactiva al perder el foco, así que el problema está en haber elegido el evento `input`
   D. El navegador conserva el mensaje hasta recargar la página; solo se soluciona reiniciando todo el formulario

Editar con WPS Office

6. En el Ejemplo 2, la función `mensajePara` ejecuta `campo.setCustomValidity('')` antes de consultar `campo.validity.valid`. ¿Qué propósito tiene esa primera instrucción?

   A. Hacer que `validationMessage` devuelva el texto en el idioma de la página en lugar del idioma del navegador
   B. Forzar que el navegador muestre de nuevo la burbuja en el siguiente intento de envío del formulario
   C. Borrar el valor escrito por la persona para evitar que se arrastren datos de intentos anteriores
   D. Evitar que un `customError` previo haga inválido el campo, de modo que `valid` refleje solo las reglas nativas

7. En el Ejemplo 2, la persona escribe `ab#` en el campo `usuario`, que declara `minlength="4"` y `pattern="[A-Za-z0-9_]+"`. Con el arreglo `ORDEN` definido (`valueMissing`, `typeMismatch`, `patternMismatch`, `tooShort`, …), ¿qué mensaje propio se muestra y por qué?

   A. “Usa solo letras, números y guion bajo (_).”, porque `patternMismatch` aparece antes que `tooShort` en `ORDEN`
   B. “El usuario debe tener al menos 4 caracteres.”, porque `tooShort` es la restricción con mayor prioridad
   C. Ambos mensajes a la vez, porque `ORDEN.find` devuelve todas las propiedades que coinciden con el error
   D. El mensaje por defecto del navegador, porque ninguna propiedad activa tiene un mensaje propio definido

8. Un sistema académico exige que el código de matrícula tenga tres letras y cuatro números (por ejemplo, `ABC1234`) y lo valida con `pattern`. Aplicando los criterios de redacción de mensajes del documento, ¿cuál es el mejor texto para el caso `patternMismatch`?

   A. “Formato inválido: el patrón no coincide con la expresión regular del campo.”
   B. “¡ERROR! Escribiste mal la matrícula, revisa lo que digitaste e inténtalo otra vez.”
   C. “Escribe la matrícula con tres letras y cuatro números, por ejemplo ABC1234.”
   D. “Valor fuera de rango en el campo de matrícula; corrige los datos ingresados.”

9. Una oficina registra a diario cientos de fichas de beneficiarios y necesita que la validación no interrumpa el ritmo de digitación. ¿Qué estrategia de momentos de validación es la más coherente con las pautas del documento?

   A. Validar al salir de cada campo (`blur`), revalidar con `input` solo tras un primer error y revisar todo al enviar
   B. Validar con cada pulsación desde el primer carácter, para que el error aparezca lo antes posible
   C. Validar únicamente al enviar, para no interrumpir la digitación y mostrar todos los errores juntos
   D. Validar al cargar la página, para que el personal vea de antemano qué campos están incorrectos

Editar con WPS Office

10. Un producto necesita mostrar varios errores a la vez, darles el estilo de la marca y asociar cada uno de forma explícita con su campo. ¿Qué alternativa de presentación es la más adecuada?

    A. La burbuja nativa del navegador, porque integra todos los mensajes automáticamente y admite estilos CSS
    B. Mensajes propios junto a cada campo, ya que admiten estilo, pueden mostrarse varios a la vez y se asocian con ARIA
    C. Mensajes generados solo con CSS mediante `::before` y `content`, porque cambian según la regla incumplida
    D. Un único resumen de errores al enviar, porque sustituye la necesidad de mensajes junto a cada campo

11. Un equipo escribe `input:invalid { border-color: #b3261e; }` para marcar los errores. Al abrir la página por primera vez, ¿qué observa la persona y cómo se corrige?

    A. Nada, pues `:invalid` solo se aplica después de que la persona intenta enviar el formulario
    B. Los campos obligatorios vacíos aparecen neutros hasta que reciben el foco por primera vez
    C. Solo los campos con `pattern` aparecen en rojo; se corrige con la pseudo-clase `:required`
    D. Los campos obligatorios vacíos se ven rojos desde la carga; se corrige con `:user-invalid` o con una clase controlada por código

12. En el Ejemplo 1, el bloque `@supports not selector(:user-invalid)` usa `placeholder=" "` junto con `:not(:placeholder-shown)`. ¿Cuál es la finalidad de esta técnica?

    A. Mostrar un texto de ayuda visible que desaparece automáticamente cuando el campo pasa a ser válido
    B. Aproximar la idea de “ya se escribió algo” en navegadores sin `:user-invalid`, para no pintar errores en campos vacíos
    C. Activar `:user-valid` en navegadores antiguos mediante el valor del atributo `placeholder` de cada campo
    D. Lograr que `:invalid` se evalúe solo tras el envío, eliminando su evaluación continua en la página

Editar con WPS Office

13. Un formulario extenso debe mostrar errores devueltos por el servidor y una regla que compara dos campos (confirmación de contraseña). ¿Qué estrategia para aplicar los estados visuales es más apropiada?

A. Solo `:user-invalid`, porque no requiere JavaScript y cubre cualquier error sin importar su origen
B. Solo `:invalid`, porque se evalúa de forma continua y así refleja también los errores del servidor
C. Clases o atributos controlados por código, como `.es-invalido` o `[aria-invalid="true"]`, más predecibles y compatibles con mensajes propios
D. Solo `:placeholder-shown`, porque indica si la persona ya interactuó con cada campo del formulario

14. Un diseñador propone que el campo erróneo se sacuda brevemente al enviar con errores y que el borde cambie de color mediante una transición. ¿Cuál es la aplicación más adecuada?

A. Mantener transiciones suaves, usar la sacudida con moderación y desactivarla con `@media (prefers-reduced-motion: reduce)`
B. Repetir la sacudida en cada pulsación para reforzar el error, pues el movimiento mejora la atención de la persona
C. Quitar el indicador de foco con `outline: none` durante la animación para evitar parpadeos visuales
D. Usar `prefers-reduced-motion` para ocultar el mensaje de error a quienes prefieren menos movimiento

15. Un formulario tiene un resumen de errores que aparece al enviar y un mensaje de progreso (“Enviando...”). ¿Qué asignación de regiones vivas sigue las recomendaciones del documento?

A. `role="alert"` para “Enviando...” y `role="status"` para el resumen, ya que el resumen no debe interrumpir
B. `role="alert"` para ambos, porque interrumpir siempre garantiza que la información se escuche
C. `aria-errormessage` para ambos, por ser el atributo con soporte más uniforme entre navegadores
D. `role="alert"` para el resumen de errores y `role="status"` para el progreso, con regiones que ya existen en el documento

Editar con WPS Office

16. En un formulario, un campo inválido cambia a borde rojo y muestra un mensaje de texto, pero una persona que usa lector de pantalla no sabe qué debe corregir. ¿Qué ajuste resuelve el problema?

A. Mover el mensaje debajo del campo en el HTML, pues el lector de pantalla lee primero el contenido siguiente
B. Subir el contraste del borde rojo a 4,5:1, porque ese valor garantiza que el lector anuncie el error
C. Vincular el mensaje con `aria-describedby`, marcar el campo con `aria-invalid="true"` y anunciar los cambios con una región viva
D. Insertar el texto con `::before` y `content`, porque el navegador lo anuncia al enfocar el campo

17. Un formulario marca los campos obligatorios con un asterisco rojo y señala los errores solo con un borde rojo. ¿Qué criterio WCAG incumple principalmente y cómo se corrige?

A. El 1.4.3, porque el contraste del texto es menor que 4,5:1; basta con oscurecer un poco el color rojo
B. El 1.4.1, porque el color es el único medio de transmitir la información; se añaden texto como “(obligatorio)” e iconos
C. El 4.1.3, porque el asterisco no recibe el foco; se corrige convirtiéndolo en un botón accesible
D. El 3.3.3, porque falta un resumen en la parte superior; se corrige agregando un `role="alert"`

18. Al registrarse, el servidor responde que el correo `ocupado@ejemplo.com` ya existe y el equipo debe presentar ese error. ¿Qué implementación es coherente con el documento?

A. Mostrar una alerta aparte del navegador con el texto recibido, para distinguirlo de los errores del cliente
B. Insertar el mensaje con `innerHTML` en un resumen general, para conservar el formato que envía el servidor
C. Mostrar el mensaje solo en el estado “Enviando...”, sin marcar el campo, para no alterar los estilos
D. Reutilizar el mecanismo del cliente: marcar el campo como inválido, escribir el texto con `textContent` en su contenedor y enfocarlo

Editar con WPS Office

19. Un equipo quiere cambiar el color de los errores y, por separado, reescribir el texto de un mensaje sin modificar la lógica ni los estilos respectivos. ¿Qué reparto de responsabilidades lo permite?

    A. HTML declara las reglas y reserva los contenedores; JavaScript decide el estado de cada campo; CSS solo representa esos estados
    B. CSS decide el estado de cada campo mediante pseudo-clases y JavaScript se limita a escribir el HTML de los mensajes
    C. JavaScript define colores y estilos en línea de cada campo, mientras CSS declara las restricciones de validación
    D. HTML contiene los mensajes y los estilos de error, y JavaScript solo se encarga de enviar el formulario

20. Un equipo afirma: “como el formulario valida todo en el navegador, no necesitamos validar en el servidor y ya cubrimos la etapa de transformación del ETL”. ¿Cómo se evalúa esta afirmación según el documento?

    A. Es correcta, porque la validación en el navegador garantiza la integridad y reemplaza la etapa de transformación
    B. Es parcialmente correcta: el servidor solo necesita validar los datos sensibles, como las contraseñas
    C. Es incorrecta: el navegador mejora la captura pero puede omitirse; el servidor valida con autoridad y no se reemplaza el ETL
    D. Es incorrecta únicamente porque los navegadores antiguos no admiten `:user-invalid` ni `:has()`

Editar con WPS Office

SECCIÓN II. RESPUESTA ABIERTA

Preguntas 21 a 25 · Explique, analice, compare o argumente con sus propias palabras.

21. Explique por qué usar `:invalid` para la retroalimentación visual genera problemas desde la carga de la página y compare esa solución con `:user-invalid` y con el uso de clases o atributos controlados por código. Indique en qué situaciones elegiría cada alternativa y por qué.

El uso de :invalid evalúa la validez de forma estática y global, provocando el fenómeno de "red-on-load" (campos marcados en rojo antes de que el usuario interactúe). Se compara con :user-invalid, que solo aplica estilos tras la interacción del usuario, y con clases controladas por código (JS), que otorgan control absoluto. :user-invalid se elige para simplicidad nativa, clases con JS para validaciones asíncronas complejas, e :invalid solo para casos globales específicos sin interacción.

22. Un compañero afirma que `setCustomValidity` “solo cambia el texto de la burbuja”. Analice esa afirmación: explique qué efectos produce realmente el método sobre el control, qué pasaría si no se limpia el mensaje y cuál es el patrón correcto para aplicarlo con los eventos `input` e `invalid`.

Es parcialmente errónea. Aunque cambia el texto de la burbuja, su efecto principal es alterar la propiedad validity.customError a true, bloqueando el envío. Si no se limpia, el control permanecerá bloqueado permanentemente aunque el valor sea correcto. El patrón correcto exige limpiar primero con setCustomValidity('') en el evento input, evaluar la condición y aplicar el mensaje si falla, usando el evento invalid para la UI.

Editar con WPS Office

23. Compare la burbuja nativa del navegador, los mensajes propios junto al campo y el resumen de errores al enviar. Argumente qué combinación recomendaría para un formulario de registro y justifique su elección con ventajas y limitaciones de cada alternativa.

La burbuja nativa es fácil pero no personalizable ni accesible; los mensajes inline ofrecen contexto inmediato pero ocupan espacio; el resumen superior da visión global al enviar. Para un registro se recomienda combinar mensajes inline en tiempo de interacción con un resumen de errores superior al enviar (focalizando al primer error y bloqueando el envío nativo), logrando la mejor combinación de accesibilidad y experiencia.

24. Explique cómo se complementan `aria-invalid`, `aria-describedby`, las regiones vivas y la gestión del foco para que un error de validación llegue a una persona que usa lector de pantalla. Argumente además por qué el color no debe ser la única señal del estado de un campo.

aria-invalid="true" avisa del error, aria-describedby conecta el campo con el texto del mensaje para anunciarlo al enfocar, las regiones vivas anuncian cambios globales sin mover el foco, y la gestión del foco traslada programáticamente al usuario al error. El color no debe ser la única señal debido a personas con daltonismo o ceguera total; debe respaldarse con iconos y texto explícito.

Editar con WPS Office

25. Analice por qué la validación en el navegador no es suficiente por sí sola. Explique qué papel cumple el servidor, cómo conviene presentar en la interfaz un error que este devuelve y cuál es la relación (y la diferencia) entre esa validación y un proceso ETL.

La validación en el navegador es eludible por los usuarios; el servidor actúa como fuente de verdad y valida reglas de negocio y unicidad. Los errores del servidor deben presentarse con el mismo mecanismo visual y ARIA que los del cliente. La diferencia con un ETL es que la validación web protege transacciones puntuales (OLTP), mientras que el ETL procesa y limpia masivamente datos por lotes para análisis estratégico (Data Warehouse).

SECCIÓN III. APLICACIÓN MEDIANTE EJEMPLOS

Editar con WPS Office

Preguntas 26 a 30 · Proponga ejemplos aplicados a situaciones reales.

26. Una entidad de salud necesita un formulario web para registrar pacientes. Proponga cuatro campos con sus atributos de restricción (por ejemplo `required`, `pattern`, `min`, `max` o `type`), indique la propiedad de `validity` que se activaría al incumplir cada uno y redacte un mensaje personalizado adecuado para cada caso.

Documento de identidad: required, pattern="[0-9]{8,10}". Validity: patternMismatch. Mensaje: "El número de documento debe contener exclusivamente entre 8 y 10 dígitos numéricos."

Edad del paciente: required, type="number", min="0", max="120". Validity: rangeUnderflow/rangeOverflow. Mensaje: "La edad ingresada no es válida; debe estar comprendida entre 0 y 120 años."

Correo electrónico: required, type="email". Validity: typeMismatch. Mensaje: "Por favor, introduzca una dirección de correo electrónico válida (ejemplo: usuario@dominio.com)."

Teléfono móvil: required, pattern="3[0-9]{9}". Validity: patternMismatch. Mensaje: "El número de teléfono móvil debe iniciar con el dígito 3 y constar exactamente de 10 dígitos."

Editar con WPS Office

27. Una tienda en línea quiere mejorar el formulario de pago de su sitio. Proponga el diseño de los estados visuales (neutro, enfocado, válido, inválido y deshabilitado) usando variables CSS, e indique qué señales distintas del color acompañarían al estado inválido. Puede incluir fragmentos breves de CSS.

Se definen variables CSS para bordes neutrales (--input-border-neutral), de enfoque (--input-border-focus), válidos (--input-border-valid) e inválidos (--input-border-invalid) usando pseudo-clases como :user-valid y :user-invalid. El estado inválido debe acompañarse de señales adicionales como iconografía (icono de advertencia) y texto descriptivo de ayuda en línea.

28. Una universidad capturará matrículas o inventarios en un panel administrativo. Describa con un ejemplo concreto en qué momentos validaría (blur, input, submit), qué haría al enviar con errores respecto al resumen y al foco, y cómo evitaría interrumpir el ritmo del personal que digita.

Se valida en input solo para restricciones inmediatas de formato, en blur al perder el foco para verificar campos completos sin interrumpir al operador, y en submit para la revisión global. Al enviar con errores, se previene el envío, se genera un resumen superior y se desplaza el foco al primer campo con error, evitando interrumpir con alertas intrusivas mientras digita.

Editar con WPS Office

29. En un sistema de información, el servidor rechaza un documento de identidad ya registrado. Proponga la estructura de la respuesta de error (campo y mensaje), describa cómo la interfaz la mostraría con el mismo mecanismo que un error del cliente y explique qué cuidado de seguridad aplicaría al insertar el texto.

Estructura JSON: {"success": false, "error": {"field": "numeroDocumento", "message": "Este número ya se encuentra registrado."}}. La interfaz lo muestra inyectando el error con el mismo mecanismo visual y ARIA que los del cliente. Para seguridad contra XSS, se debe insertar el texto usando estrictamente la propiedad textContent.

30. Una empresa consolida en un proceso ETL los datos que sus clientes escriben en un formulario web de pedidos. Plantee un ejemplo con reglas concretas en el navegador y en el servidor, y explique qué aporta cada ámbito a la calidad de los datos y por qué ninguno reemplaza las etapas del ETL.

Reglas: En el navegador (cliente) se valida el formato de código postal (pattern) y cantidades mínimas; en el servidor se verifica la existencia del producto en la base de datos y la sesión. El navegador aporta experiencia fluida y ahorro de ancho de banda; el servidor asegura la seguridad transaccional e integridad. Ninguno reemplaza al ETL porque el ETL limpia, normaliza y consolida masivamente datos históricos para análisis, a diferencia de la validación unitaria transaccional.

Editar con WPS Office